

# Introduction to Deep Learning

## Session 10: CNNs in Practice

May 2026

Magda Gregorová - [magda.gregorova@thws.de](mailto:magda.gregorova@thws.de)



Licensed according to CC-BY-SA 4.0

# Lecture Overview

## 1. The Sequence Problem

## 2. Vanila RNNs

# The Sequence Problem

Many real-world inputs are **ordered and variable-length** — their meaning depends on position and context.

## Sequence inputs

- Natural language: “*The food was not good*”
- Speech: 16 000 samples/second
- Time series: stock prices, sensor readings
- Video: ordered frames

## Key properties

- **Variable length** — no fixed input dimension
- **Order matters** — permuting changes meaning
- **Long-range dependencies** — word at step 1 may determine meaning at step 50

**We need a model that processes sequences step-by-step and accumulates context.**

# Three Approaches to Sequence Modeling

|                           | MLP | 1D CNN      | RNN          |
|---------------------------|-----|-------------|--------------|
| Handles variable length   | no  | no*         | yes          |
| Long-range dependencies   | no  | limited     | yes          |
| Parallelizable training   | yes | yes         | no           |
| Parameter sharing in time | no  | yes (local) | yes (global) |

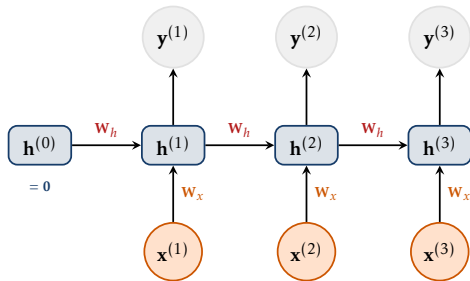
\*requires padding or global pooling

**1D CNNs** — fast, parallelizable, strong for **local patterns** (n-grams, short-term trends); receptive field bounded by depth. Useful when long-range context not needed.

**RNNs** — process step-by-step; **unbounded context** via hidden state; sequential by design.

**This session focuses on RNNs.**

# The Recurrence Idea



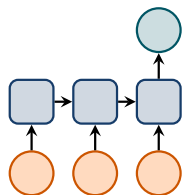
At each timestep  $t$ , compute **hidden state**  $h^{(t)}$  from current input and previous state:

$$h^{(t)} = f(x^{(t)}, h^{(t-1)}), \quad h^{(0)} = 0$$

Same  $f$ , same weights  $w_h, w_x$  at every timestep — parameter sharing across time.

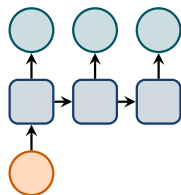
# Task Shapes

The same recurrent architecture supports **four input/output configurations** depending on where outputs are read:



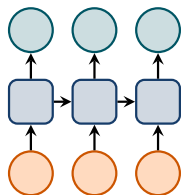
**Many-to-one**  
*sentiment analysis*

sentiment analysis  
document classification



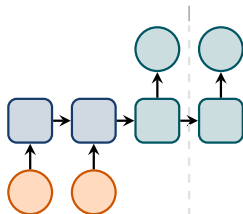
**One-to-many**  
*image captioning*

image captioning  
music generation



**Many-to-many**  
*POS tagging*

POS tagging  
named entity  
recognition

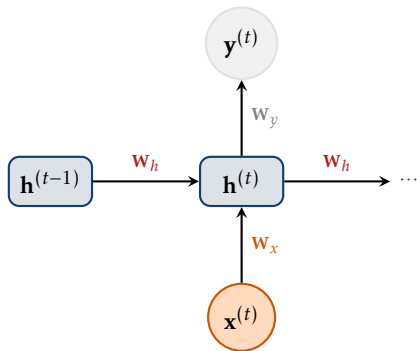


**Many-to-many**  
*machine translation*

machine translation  
speech recognition

## Vanilla RNNs

# The RNN Cell: Forward Pass



$$\mathbf{h}^{(t)} = \tanh \left( \underbrace{\mathbf{W}_h \mathbf{h}^{(t-1)}}_{\text{recurrent}} + \underbrace{\mathbf{W}_x \mathbf{x}^{(t)}}_{\text{input}} + \mathbf{b} \right)$$

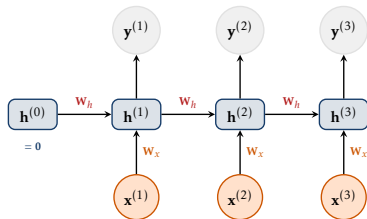
$$\mathbf{y}^{(t)} = \mathbf{W}_y \mathbf{h}^{(t)}$$

- $\mathbf{W}_h \in \mathbb{R}^{d_h \times d_h}$  — mixes previous hidden state
- $\mathbf{W}_x \in \mathbb{R}^{d_h \times d_x}$  — projects current input
- $\tanh$  squashes to  $(-1, 1)$ ; keeps  $\mathbf{h}$  bounded



# Unrolling the RNN

Unrolling reveals the **computational graph used for backpropagation**



- The same  $W_h$ ,  $W_x$ , at every node — gradients from all steps accumulate into one update
- $h^{(0)} = \mathbf{0}$  is a fixed initial condition (sometimes learned)

# Backpropagation Through Time

The gradient of the loss with respect to  $\mathbf{W}_h$  involves a **product of  $T$  Jacobians**:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} \propto \prod_{t=1}^T \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}}$$

Each factor involves  $\mathbf{W}_h$  and the derivative of  $\tanh$  (which is  $\leq 1$ ). **Multiplying  $T$  small numbers  $\rightarrow$  gradient vanishes.**